

Universidad Politécnica Internacional

Técnicas de Programación
Documentación Técnica

Sistema de Gestión de Quinielas Mundialistas
“Quiniela Mundial V2”

Estudiante:
Glen Gerardo Villagra Rodríguez

Profesor:
Luis Felipe Mora Umaña

Segundo Cuatrimestre del 2026

1.Introducción	2
2.Decisiones de diseño y arquitectura.....	2
2.1. Patrón Arquitectónico y Capas (MVC).....	3
2.2. Persistencia de Datos Relacional.....	3
2.3. Control de Usuarios y Borrado Lógico.....	3
2.4. Gestión de Estado y Roles	3
2.5. Exportación de Reportes.....	3
3.Diagrama de Bases de Datos.....	4
Diagrama de base de datos relacional	4
Estructura de Tablas Principal:	4
4.Desarrollo y Guia de ejecución.....	4
4.1. Requisitos del Entorno.....	5
4.2. Flujo de Operación por Módulo	5
5.Cobertura de pruebas unitarias (Testing).....	5
6.Análisis de resultados aprendizaje y conclusiones	6
6.1. Análisis de Resultados.....	6
6.2. Aprendizajes Obtenidos.....	6
6.3. Conclusiones.....	7

1.Introducción

El presente informe expone la evolución técnica y funcional del Sistema de Gestión de Quinielas Mundialistas en su segunda versión (v2), desarrollado sobre la plataforma .NET utilizando C#. El propósito principal de esta iteración es llevar el software hacia un entorno web robusto, sustituyendo la persistencia local por una base de datos relacional y migrando la capa gráfica previa hacia la tecnología **Blazor Web**.

Esta versión amplía la capacidad del sistema introduciendo control de acceso basado en roles (Administrador y Usuario), un módulo de alertas preventivas para partidos pendientes en las próximas 24 horas, un panel de estadísticas extendido con capacidad de exportación de reportes y la integración de pruebas unitarias para asegurar la calidad de la lógica de negocio.

2.Decisiones de diseño y arquitectura

El desarrollo se fundamenta en la simplificación del código y la claridad algorítmica, priorizando el uso de estructuras de control fundamentales (if, foreach, listas genéricas List<T>) para garantizar un código limpio, legible y de fácil comprensión para esta segunda iteración.

2.1. Patrón Arquitectónico y Capas (MVC)

Con el objetivo de mantener una arquitectura directa y sin abstracciones complejas, el proyecto se dividió en:

Vistas (View - Blazor Web): Componentes Razor dedicados únicamente a la presentación visual y captura de datos en el navegador web, reemplazando la interfaz anterior de Windows Forms.

Modelos (Models): Clases sencillas que representan las entidades del sistema (Usuario, Partido, Pronóstico, Insignia) conteniendo únicamente sus atributos esenciales.

Lógica de Negocio y Datos: Métodos directos en C# que procesan las validaciones, cálculos de puntos y consultas mediante el uso de condicionales y ciclos básicos, omitiendo el uso de patrones de diseño complejos o capas intermedias de servicios innecesarias.

2.2. Persistencia de Datos Relacional

Se sustituyó de forma integral la lectura/escritura de archivos planos (.csv) por una Base de Datos Relacional. Las transacciones de información (consultas, registros y actualizaciones) se

ejecutan de manera directa, garantizando la integridad referencial entre las tablas de Usuarios, Partidos, Pronósticos e Insignias.

2.3. Control de Usuarios y Borrado Lógico

Para la administración de usuarios, se adoptó la política de desactivación lógica (Activo = false) en lugar de la eliminación física de los registros en la base de datos. Esta decisión protege la integridad histórica de las tablas de pronósticos y estadísticas generadas por dichos usuarios.

2.4. Gestión de Estado y Roles

El control de sesión se gestiona identificando dos roles definidos:

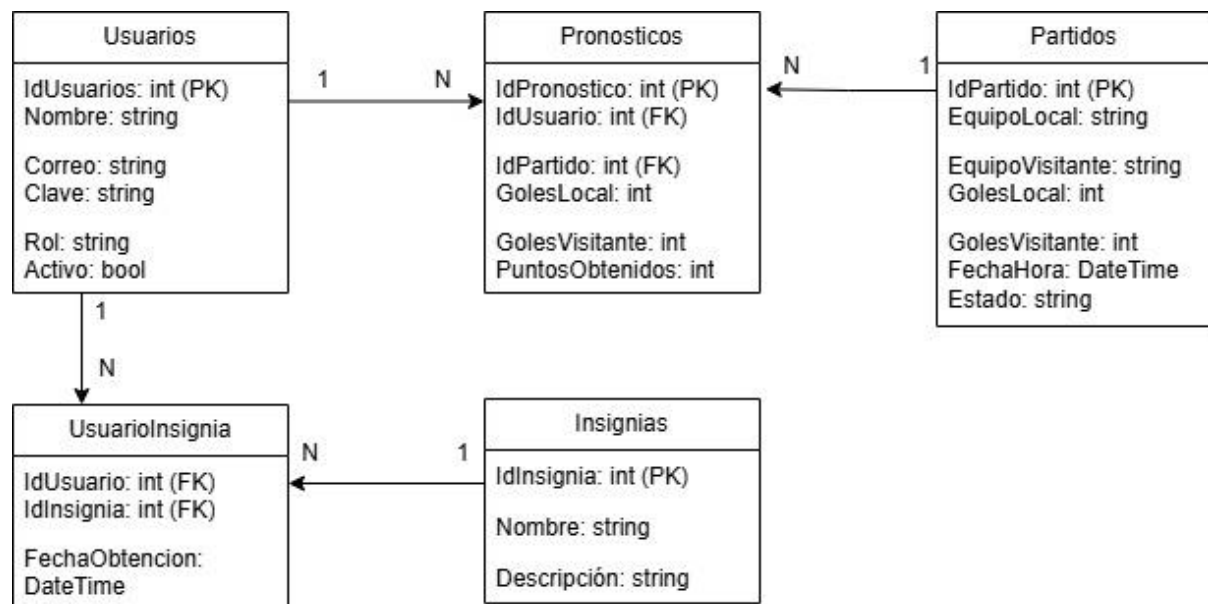
1. **Administrador:** Acceso exclusivo a la gestión de partidos (creación, actualización de marcadores, fechas y horas) y mantenimiento de usuarios (desactivación y reestablecimiento de contraseñas). Queda inhabilitado para registrar pronósticos.
2. **Usuario Regular:** Acceso al dashboard de pronósticos, notificaciones de partidos dentro de las próximas 24 horas, consulta de insignias, ranking y reportes personales.

2.5. Exportación de Reportes

Se incorporó un módulo de generación de archivos de texto en formato .txt / .csv utilizando la clase StreamWriter. Los reportes consolidados (tanto de administración como de usuarios) son procesados y convertidos a cadenas de texto para su descarga directa desde la interfaz web.

3.Diagrama de Bases de Datos

Diagrama de base de datos relacional



Estructura de Tablas Principal:

Usuarios: IdUsuario (PK), Nombre, Correo, Clave, Rol (Admin/Usuario), Activo (Boolean).

Partidos: IdPartido (PK), EquipoLocal, EquipoVisitante, GolesLocal, GolesVisitante, FechaHora, Estado.

Pronosticos: IdPronostico (PK), IdUsuario (FK), IdPartido (FK), GolesLocal, GolesVisitante, PuntosObtenidos.

Insignias: IdInsignia (PK), Nombre, Descripcion.

UsuarioInsignia: IdUsuario (FK), IdInsignia (FK), FechaObtencion.

4.Desarrollo y Guia de ejecución

4.1. Requisitos del Entorno

- IDE: Visual Studio con la carga de trabajo de Desarrollo Web y ASP.NET.
- Runtime: .NET SDK actualizado.
- Motor de Base de Datos: Servidor SQLite configurado.

4.2. Flujo de Operación por Módulo

Autenticación e Inicio de Sesión: El usuario ingresa sus credenciales en la pantalla de Login en Blazor. El sistema verifica mediante condicionales que el usuario exista y que su estado Activo sea true.

Notificaciones (Inicio de Sesión - Usuario): Al ingresar el usuario, el sistema evalúa las fechas comparando la hora actual contra la hora programada de los encuentros ($\text{FechaHora} - \text{DateTime.Now} \leq 24$ horas). Si existen partidos pendientes de pronóstico, se despliega una alerta en pantalla.

Mantenimiento (Administrador): El administrador puede modificar la fecha, hora y marcadores finales de cualquier encuentro. Al finalizar un partido, el sistema ejecuta el algoritmo de cálculo de puntos para actualizar las posiciones de los usuarios de forma inmediata.

Registro de Pronósticos (Usuario): Los usuarios seleccionan los partidos disponibles y guardan sus marcadores proyectados.

Módulo de Reportes y Descargas: Se habilitan pantallas de consulta según el rol:

Métricas Admin: Resultado más repetido, partido con más aciertos, usuarios con más aciertos (Top 1, 3, 5), promedio de goles, partidos sin aciertos.

Métricas Usuario: Equipo más apostado, equipo sorpresa y probabilidad de aciertos en pronósticos anteriores.

Cada reporte incluye un botón interactivo para exportar el contenido a formato de texto plano (.txt o .csv).

5. Cobertura de pruebas unitarias (Testing)

Para validar la solidez del sistema, se estructuró un proyecto de pruebas unitarias sobre los métodos de lógica principal, cubriendo al menos el 60% de la funcionalidad.

Casos de prueba validados:

Asignación correcta de puntajes (+5 por resultado exacto, +2 por tendencia).

Restricción de registro de pronósticos sobre partidos expirados.

Correcto filtrado de notificaciones en la ventana de 24 horas.

Comprobación del estado Activo durante el inicio de sesión.

6. Análisis de resultados aprendizaje y conclusiones

6.1. Análisis de Resultados

1. **Migración Exitosa a Entorno Web:** Se completó la transición desde la interfaz de escritorio en Windows Forms hacia la plataforma **Blazor Web**, logrando una aplicación web liviana y funcional que responde correctamente a las solicitudes de los usuarios a través del navegador.
2. **Control Eficiente de Roles y Acceso:** La implementación de la lógica de autenticación permitió separar adecuadamente las atribuciones del rol Administrador y del rol Usuario Regular, garantizando que las funciones de edición de partidos y usuarios queden restringidas únicamente al perfil administrativo.
3. **Persistencia e Integridad de Datos Relacionales:** La sustitución de archivos .csv por una estructura de base de datos relacional resolvió las inconsistencias de lectura/escritura simultánea, asegurando que los pronósticos, puntuaciones e insignias se sincronicen de manera inmediata en todas las pantallas.

6.2. Aprendizajes Obtenidos

1. **Beneficios del Desecho Lógico (Activo = false):** Se aprendió que desactivar registros en lugar de eliminarlos físicamente es una práctica fundamental en sistemas transaccionales, ya que permite suspender el acceso a usuarios sin romper las relaciones en las tablas de pronósticos ni alterar las métricas históricas.
2. **Ventajas de la Lógica Simplificada:** Se comprobó que utilizar estructuras de control fundamentales (if, foreach, listas genéricas List<T>) en lugar de capas de servicios o patrones complejos reduce drásticamente la presencia de código muerto y facilita la rastreabilidad de errores durante la ejecución.
3. **Validación Mediante Pruebas Unitarias:** La creación de pruebas sobre los métodos de lógica de negocio enseñó la importancia del *testing* automatizado, permitiendo verificar que los algoritmos de cálculo de puntos y notificaciones funcionen de forma correcta antes de desplegar la interfaz gráfica.

6.3. Conclusiones

1. **Mantenibilidad y Sencillez del Código:** Mantener una arquitectura directa y sin abstracciones innecesarias resultó ser la decisión más adecuada para el proyecto, ya que garantiza que el código fuente sea claro, legible y 100% defendible durante una evaluación técnica.

2. **Adaptabilidad de Blazor Web:** La tecnología Blazor demostró ser una alternativa sumamente ágil dentro del ecosistema .NET, permitiendo reutilizar el conocimiento previo en lenguaje C# para construir interfaces web dinámicas sin requerir librerías externas complejas.
3. **Cumplimiento de los Objetivos de la Iteración 2:** El sistema incrementó de forma satisfactoria su nivel de madurez al cumplir con los requisitos de la nueva versión soporte de roles, alertas en tiempo real, exportación de reportes a texto plano y cobertura de pruebas unitarias superior al mínimo requerido.